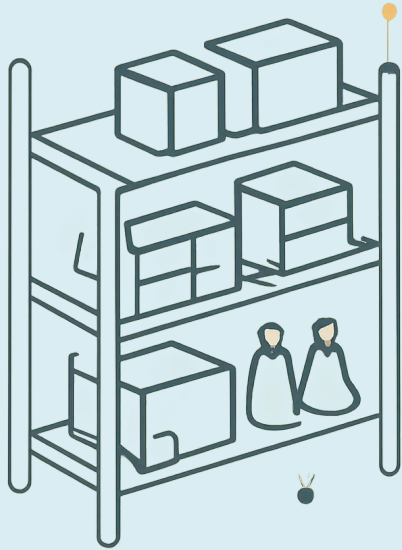




Gestión Eficiente de Inventario

Implementación de algoritmos **Merge Sort** y **Búsqueda Binaria** para la organización y búsqueda de productos en una tienda.

PROYECTO FINAL · ALGORITMIA

El Problema: Inventario Desordenado

En una tienda real, los productos se registran **a medida que llegan**, sin ningún orden predefinido. Los códigos numéricos quedan dispersos, lo que convierte la búsqueda de un artículo concreto en un proceso **lento e ineficiente**.

Registro caótico

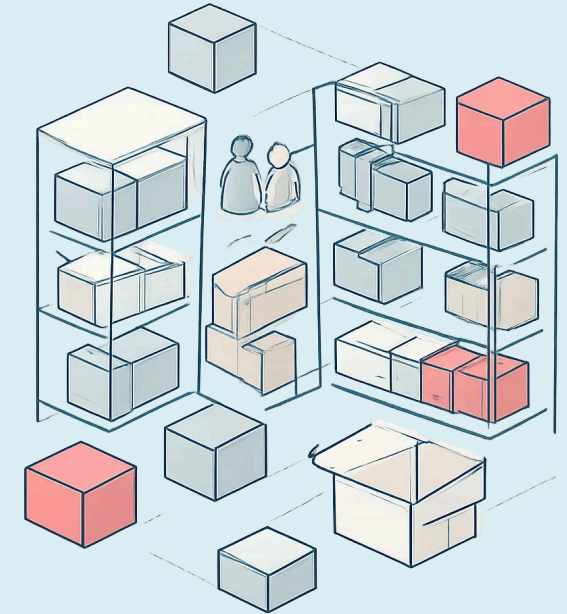
Los códigos de producto se almacenan en el orden de llegada, sin criterio de ordenación.

Búsqueda lineal costosa

Localizar un artículo requiere revisar la lista completa, elemento a elemento.

Escalabilidad nula

A mayor número de productos, mayor el tiempo de respuesta, haciéndolo inviable en grandes inventarios.



La Solución Propuesta

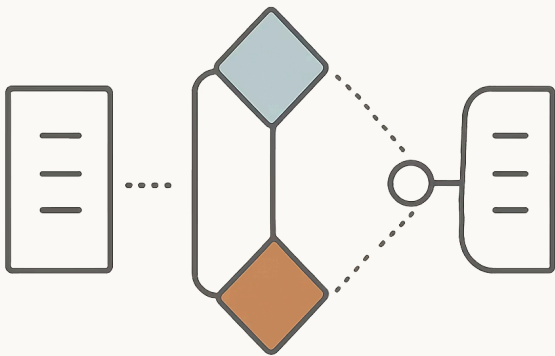
Se desarrolló un programa en **Python** estructurado en **dos fases secuenciales** que transforman un inventario caótico en un sistema de gestión ágil y preciso.



La combinación de ambos algoritmos garantiza **eficiencia máxima**: primero se organiza el inventario y después se localiza cualquier producto en tiempo mínimo.

Ordenamiento con Merge Sort

ESTRATEGIA: DIVIDIR Y CONQUISTAR



Merge Sort es un algoritmo de ordenamiento estable y eficiente que opera dividiendo recursivamente la lista hasta obtener subconjuntos de un solo elemento y luego los combina ordenadamente.

01

Dividir

Se parte la lista en dos mitades iguales de forma recursiva.

02

Conquistar

Cada mitad se ordena de forma independiente mediante llamadas recursivas.

03

Combinar

Las mitades ordenadas se fusionan en una única lista completamente ordenada.

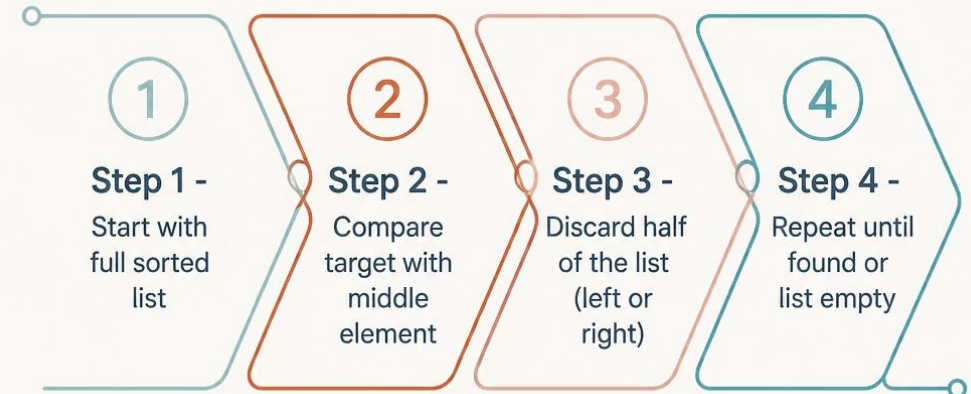
Búsqueda Binaria

SOLO FUNCIONA SOBRE LISTAS ORDENADAS

Una vez ordenado el inventario con Merge Sort, la **Búsqueda Binaria** localiza cualquier código de producto de forma extremadamente rápida.

El algoritmo compara el código buscado con el elemento central de la lista y **descarta la mitad de las opciones** en cada iteración, repitiendo el proceso hasta encontrar el elemento o determinar que no existe.

- ❏ Cada paso reduce el espacio de búsqueda a la mitad: con 1.000 productos, se necesitan como máximo **10 comparaciones**.





Conclusiones

Este proyecto demuestra cómo la aplicación de **estructuras de datos y algoritmos correctos** transforma un problema logístico real en una solución de software escalable, rápida y eficiente.



Solución práctica

Python + Merge Sort + Búsqueda Binaria resuelven un problema real de gestión de inventario.



Eficiencia garantizada

Complejidades $O(n \log n)$ y $O(\log n)$ aseguran rendimiento óptimo incluso con miles de productos.



Escalabilidad

El sistema crece con la tienda sin degradación significativa del tiempo de respuesta.